Instagram Feed Ranking

High-level ML system design — data platform, ML platform, models, and online feed generation

---

0. Problem Framing

Feed generation is not *"SELECT * FROM posts ORDER BY created_at DESC."* The real problem: given user U at time T, produce the best ordered set of content from a candidate universe of billions of items, under latency, safety, diversity and business constraints. The pipeline is a funnel: **Candidate generation** → **Filtering** → **Ranking** → **Re-ranking** → **Top N**, and it runs across four conceptual layers.

Layer	Name	Role
Layer 4	Data / Event Platform	Capture every user interaction as a signal, in real time and in bulk
Layer 5	ML Platform	Turn raw events into features, embeddings, trained models, and a way to serve/monitor them
Layer 6	AI / ML Models	Retrieval, first-stage ranking, second-stage ranking, integrity
Layer 7	Online Feed Generation	Combine model scores, apply re-ranking rules, return the final feed

1. Layer 4 — Data / Event Platform

Every action — impression, like, comment, share, save, skip, hide, watch-time, profile visit — becomes an **InteractionEvent** (user_id, post_id, event_type, timestamp, session_id, position, surface). This layer splits into two paths that run at different speeds:

Streaming path (what just happened)

Captures short-term intent — last few clicks, current session, recent likes. Latency: milliseconds to minutes. Needed because waiting for a nightly batch job to notice "user just watched 10 boxing reels" is too slow.

Batch path (what has happened historically)

Captures long-term interest — 30-day engagement, creator affinity, embeddings, training datasets. Latency: hours to days.

Technology / Technique	Why it's used here
Distributed event bus (Kafka-like)	Mobile clients generate huge event volume continuously; a durable, distributed log lets many downstream consumers (stream processor, data lake) read independently without blocking the client.
Stream processor	Converts raw events into "recent interest" features within seconds, so the very next feed request reflects what the user just did.
Data lake / warehouse	Cheap, durable storage for the full historical event log; source of truth for training datasets and offline analytics.

Note: Meta doesn't publicly confirm Kafka specifically for this pipeline — in an interview, frame it as "I'd use Kafka/PubSub-style streaming; Meta's exact internal event bus is proprietary." That distinction itself signals seniority.

2. Layer 5 — ML Platform

Raw events alone aren't usable by a model. This layer turns them into features, embeddings, trained models, and the infrastructure to serve and monitor all of it.

Feature store (online vs offline)

A feature like *user_creator_affinity* shouldn't be recomputed from raw events on every request — it's precomputed once and looked up. The **offline store** serves training/backtesting (historical, computed in batch). The **online store** serves live inference (low latency, high QPS, e.g. Redis / Cassandra / DynamoDB-style KV store). You can't store a dense user × creator matrix at 1B users × millions of creators — hence sparse features, aggregation, and embeddings instead of raw lookups.

Two-Tower embeddings

A user tower and an item tower each produce an embedding; similarity between them approximates relevance. This avoids the impossible alternative of running a full network(user, item) pair for every one of billions of items at request time. Item embeddings are **precomputed offline** and stored in an ANN index; only the user embedding is computed fresh online, then matched via approximate nearest neighbor search.

Technology / Technique	Why it's used here
Online feature store (Redis / KV-store class)	Sub-millisecond feature lookups during live ranking; can't afford a DB query per feature per candidate.
Offline feature pipelines + data lake	Batch-compute expensive, slow-changing features (30-day engagement, affinity) without touching the online path.
Two-Tower model + ANN index (FAISS/HNSW-class)	Makes retrieval over billions of items computationally feasible — precompute once, search fast.
Model registry	System of record for 1000+ production models: version, owner, surface, criticality — so a quality regression can be traced to "which model changed."
Model monitoring (calibration, normalized entropy)	A model can look healthy on infra metrics (200 OK, low latency) while badly miscalibrated on predictions — ML reliability is a distinct concern from service uptime.

3. Layer 6 — AI / ML Models (the funnel)

You cannot run one heavy model against a billion candidates. The funnel narrows the set at each stage, spending more compute only on candidates that survived the cheaper stage before it:

1B candidates → **Retrieval** → **~10K** → **First-stage rank** → **~500** → **Second-stage rank** → **~100** → **Re-rank** → **~20**

Retrieval (Model A)

Combines multiple sources with tunable weights, not just one method: social graph (who you follow), Two-Tower ANN similarity (embedding match to things you've engaged with), and trending/popular content (cold-start fallback for new users with no history).

First-stage ranking (Model B)

~10K → ~500. Must be cheap, so it reuses the cacheable Two-Tower architecture (it deliberately avoids user-item interaction features, which would break cacheability). It's trained via **knowledge distillation**: the heavier second-stage model acts as "teacher," and the lightweight first-stage model is trained to approximate the teacher's top picks — giving speed with reasonable accuracy.

Second-stage ranking (Model C) — MTML

~500 → ~100. Now it's affordable to run a heavy **Multi-Task Multi-Label (MTML)** deep network. A shared representation feeds several prediction heads at once — P(like), P(comment), P(share), P(see-less) — because a like, a comment, and a share each carry different signal; collapsing them into one label loses information.

Value Model (score combination)

The multiple task probabilities are combined into one score: **Value = w_{like}·P(like) + $w_{comment}$·P(comment) + w_{share}·P(share) − w_{see_less}·P(see_less)**. The weights are a product decision, not a modeling one — this is exactly where "engagement vs. user wellbeing" tradeoffs get encoded, and where Bayesian/offline optimization is used to tune weights instead of hand-guessing them.

Technology / Technique	Why it's used here
Graph retrieval / GNN-style signals	Strong for new users and new creators where embedding history is sparse or absent.
Two-Tower + ANN (FAISS/HNSW)	Only computationally feasible way to search billions of items in milliseconds.
Knowledge distillation (first-stage from second-stage)	Gets most of the heavy model's judgment at a fraction of the inference cost, at the ~10K-candidate scale.
MTML deep network (second-stage)	One shared model learns correlated behaviors jointly, improving generalization vs. training isolated single-label models.
Online / exploration-aware learning	New content with zero engagement history would never surface under pure popularity scoring; exploration gives it a fair chance to find its audience.

4. Layer 7 — Online Feed Generation & Re-ranking

The Value Model's ranked list is not the final feed. A re-ranking pass applies rules on top of ML scores: **diversity** (don't show 5 posts from the same author back-to-back), **integrity/safety demotions** (policy-violating content pushed down regardless of predicted engagement), and **freshness/business rules** (ad slots, recency floors). This is deliberately rule-based on top of ML, not another giant model — some constraints (safety, ad load) are business-critical and shouldn't be left purely to a learned score.

Precomputation matters at this stage too: recommendations for some candidate sources can be generated offline during off-peak hours rather than fully online for every request — a hybrid of precomputation + online retrieval + online ranking + final re-ranking, not a pure real-time system end to end.

5. The Feedback Loop

The feed shown to the user generates new interaction events, which flow back into the event stream, split again into real-time features (immediately usable for the next ranking request) and batch training data (used to retrain models, sometimes hourly). Retrained models are deployed through the model registry with capacity/load testing before rollout — new models don't go live without an estimate of QPS, latency, and error-rate impact. This closed loop is what separates a recommender system from a static database query.

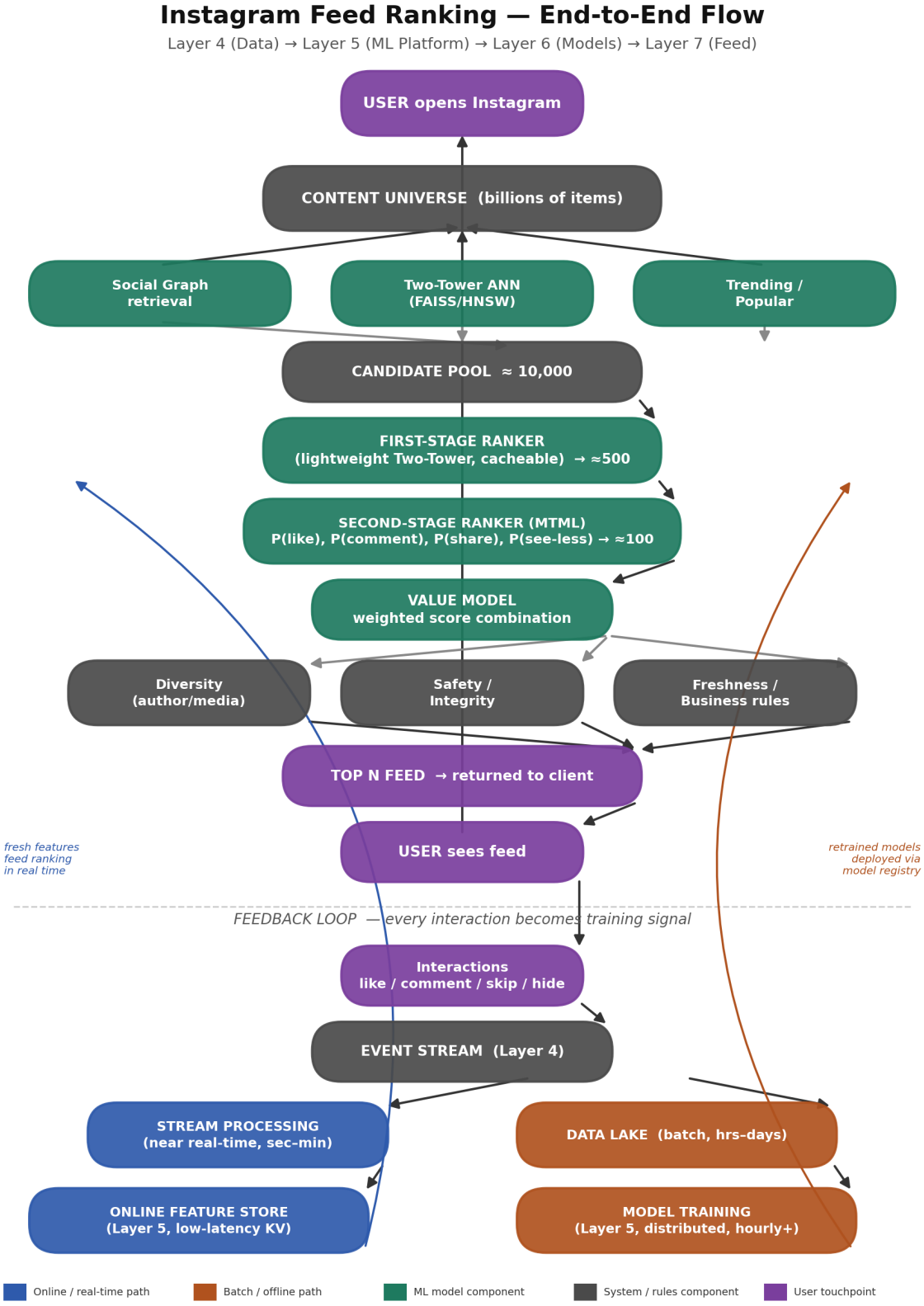
6. What's Meta-Confirmed vs. Interview-Reasonable

Useful discipline for an interview: separate what Meta has publicly described from what you're proposing as a reasonable implementation.

Category	Examples

Publicly confirmed	Multi-stage retrieval/ranking funnel; multiple retrieval sources; Two-Tower retrieval; offline item-embedding generation + online ANN (FAISS/HNSW cited as examples); MTML second-stage model; Value-Model-style score combination; final integrity/diversity re-ranking; 1000+ production ML models by 2025; model registry; hourly retraining (2023 Explore article); Bayesian/offline parameter tuning.
Reasonable interview architecture	Kafka, Spark, Flink, Redis, Cassandra, Kubernetes, Feast, Airflow, MLflow — propose these as "what I would use," not as confirmed Meta internals.
Avoid claiming	Exact QPS numbers, exact embedding dimensions, specific databases as "Instagram's tech" — unless the source actually states it.

7. End-to-End Flow Diagram



One feed request, traced end to end: retrieval → first-stage → second-stage MTML → value model → re-ranking → delivery, and the feedback loop that turns every interaction back into training signal.